

# Introduction to Large Language Models

## Week 1

Transformers: Multi-Headed Self-Attention & Cross-Attention

Detailed study notes with intuition, derivations and a large practice bank

Based on lectures by Prof. Mitesh M. Khapra (AI4Bharat, IIT Madras)

### How to use these notes

1. Read Part I (theory) in order: each idea builds on the previous one.
2. Yellow *Intuition* boxes explain *why*; blue boxes give exact definitions; red boxes flag common exam traps.
3. Part II is the question bank. Attempt each question before reading its answer.
4. Every numerical answer has been checked by running the computation.

## Contents

|                                                                                                             |           |
|-------------------------------------------------------------------------------------------------------------|-----------|
| <b>Part I — Theory</b>                                                                                      | <b>2</b>  |
| <b>1 Prerequisites You Need Before Starting</b>                                                             | <b>2</b>  |
| 1.1 Vectors, dot products and similarity . . . . .                                                          | 2         |
| 1.2 Matrix–vector and matrix–matrix products . . . . .                                                      | 2         |
| 1.3 The softmax function . . . . .                                                                          | 2         |
| 1.4 Word embeddings . . . . .                                                                               | 3         |
| 1.5 Recurrent Neural Networks (RNNs) . . . . .                                                              | 3         |
| <b>2 Module 1.1 — Limitations of Sequential Models</b>                                                      | <b>4</b>  |
| 2.1 The sequence-to-sequence (seq2seq) model . . . . .                                                      | 4         |
| 2.2 Attention: a quick tour . . . . .                                                                       | 5         |
| 2.3 The major limitation . . . . .                                                                          | 6         |
| <b>3 Module 1.2 — Attention Is All You Need</b>                                                             | <b>7</b>  |
| 3.1 Transition from RNN seq2seq to Transformers . . . . .                                                   | 7         |
| 3.2 Self-attention . . . . .                                                                                | 7         |
| 3.3 Choice of attention (score) function . . . . .                                                          | 8         |
| 3.4 Transformation matrices: query, key, value . . . . .                                                    | 9         |
| 3.5 Computing the first output $z_1$ , step by step . . . . .                                               | 10        |
| 3.6 Vectorising: all outputs in one go . . . . .                                                            | 10        |
| 3.7 Multi-head attention . . . . .                                                                          | 12        |
| 3.8 The encoder layer . . . . .                                                                             | 13        |
| 3.9 The encoder stack . . . . .                                                                             | 15        |
| 3.10 Cross-attention (encoder–decoder attention): preview . . . . .                                         | 16        |
| <b>4 Big-Picture Comparison and Formula Sheet</b>                                                           | <b>17</b> |
| 4.1 RNN seq2seq with attention vs. Transformer encoder . . . . .                                            | 17        |
| 4.2 Formula sheet . . . . .                                                                                 | 17        |
| 4.3 Shape cheat-sheet ( $d_{\text{model}} = 512$ , $h = 8$ , $d_k = d_v = 64$ , $d_{ff} = 2048$ ) . . . . . | 18        |
| <b>Part II — Question Bank</b>                                                                              | <b>19</b> |
| <b>5 Numerical Problems (with full worked solutions)</b>                                                    | <b>19</b> |
| <b>6 Subjective / Descriptive Questions</b>                                                                 | <b>26</b> |
| <b>7 Multiple Choice Questions (single correct)</b>                                                         | <b>31</b> |

|                                                          |           |
|----------------------------------------------------------|-----------|
| <b>8 Multiple Select Questions (one or more correct)</b> | <b>45</b> |
| <b>9 True or False (with justification)</b>              | <b>49</b> |
| <b>10 Fill in the Blanks</b>                             | <b>51</b> |
| <b>11 Quick Self-Check Before the Exam</b>               | <b>51</b> |

# Part I — Theory

## 1 Prerequisites You Need Before Starting

The lecture assumes some background. These are recapped here so that no step later feels like a jump.

### 1.1 Vectors, dot products and similarity

For two vectors  $\mathbf{a}, \mathbf{b} \in \mathbb{R}^d$  the **dot product** is

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \sum_{i=1}^d a_i b_i = \|\mathbf{a}\| \|\mathbf{b}\| \cos \theta.$$

#### Intuition

The dot product is large and positive when two vectors point in a similar direction, close to zero when they are perpendicular (unrelated), and negative when they point in opposite directions. That is why attention uses dot products as a **similarity score**: “how related are these two words?” becomes “how aligned are their vectors?”

### 1.2 Matrix–vector and matrix–matrix products

If  $W \in \mathbb{R}^{m \times n}$  and  $\mathbf{h} \in \mathbb{R}^n$ , then  $W\mathbf{h} \in \mathbb{R}^m$ . A matrix is a **linear transformation**: it maps a vector from one space ( $n$  dimensions) to another ( $m$  dimensions).

If  $A \in \mathbb{R}^{m \times n}$  and  $B \in \mathbb{R}^{n \times p}$ , then  $AB \in \mathbb{R}^{m \times p}$ . The inner dimensions must match; the outer dimensions give the result’s shape. **Always check shapes** — it is the fastest way to catch mistakes in this course.

#### Be Careful

Stacking vectors as *columns* of a matrix  $H = [\mathbf{h}_1, \dots, \mathbf{h}_T]$  and then computing  $WH$  applies  $W$  to every column at once:  $WH = [W\mathbf{h}_1, W\mathbf{h}_2, \dots, W\mathbf{h}_T]$ . This single idea is what makes Transformers parallel.

### 1.3 The softmax function

For a vector of real scores  $\mathbf{e} = (e_1, \dots, e_n)$ ,

$$\text{softmax}(\mathbf{e})_j = \frac{\exp(e_j)}{\sum_{j'=1}^n \exp(e_{j'})}.$$

Properties you must know:

1. Every output is strictly positive:  $\text{softmax}(\mathbf{e})_j > 0$ .
2. The outputs sum to 1, so they form a probability distribution.
3. It preserves order: a larger score gets a larger probability.
4. **Shift invariance**:  $\text{softmax}(\mathbf{e} + c) = \text{softmax}(\mathbf{e})$  for any constant  $c$  (the  $e^c$  cancels in numerator and denominator).

5. **Not** scale invariant: multiplying scores by a number  $> 1$  makes the distribution sharper (peakier); dividing makes it flatter (more uniform).
6. When one score is much larger than the rest, softmax is almost one-hot and its gradient is almost zero. This is called **saturation**.

### Intuition

Softmax is a “soft” version of argmax. Argmax says “pick the single best”. Softmax says “give most weight to the best but keep some weight for the others”. Because it is smooth, we can differentiate through it and train with gradient descent.

## 1.4 Word embeddings

A neural network cannot read the string “movie”. Each word is mapped to a vector  $\mathbf{h}_j \in \mathbb{R}^{d_{\text{model}}}$  called its **embedding**. In the lecture’s toy example the embeddings are 3-dimensional:

| Word      | I         | enjoyed   | the       | movie     | transformers |
|-----------|-----------|-----------|-----------|-----------|--------------|
| Embedding | [1, 0, 0] | [0, 1, 0] | [1, 1, 0] | [1, 0, 1] | [1, 1, 1]    |

In the real Transformer,  $d_{\text{model}} = 512$ .

## 1.5 Recurrent Neural Networks (RNNs)

An RNN reads a sequence one element at a time and keeps a hidden state:

$$\mathbf{h}_t = f(W\mathbf{h}_{t-1} + U\mathbf{x}_t + \mathbf{b}), \quad \mathbf{h}_0 = \mathbf{0} \text{ (typically).}$$

The hidden state  $\mathbf{h}_t$  is a running summary of  $x_1, \dots, x_t$ . The crucial property:  **$\mathbf{h}_t$  cannot be computed until  $\mathbf{h}_{t-1}$  is known**. This recursive dependence is the source of both the RNN’s power and its biggest weakness (Module 1.1).

## 2 Module 1.1 — Limitations of Sequential Models

### 2.1 The sequence-to-sequence (seq2seq) model

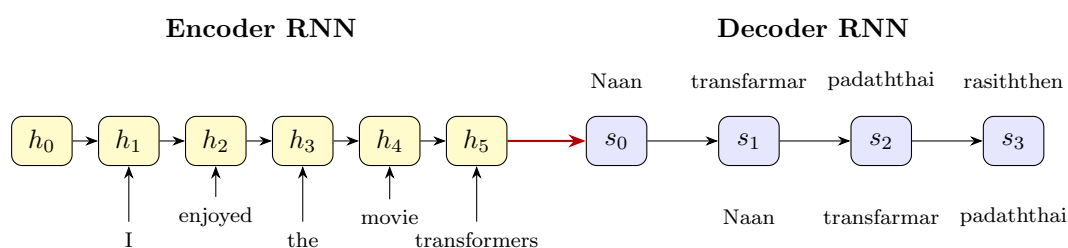
Task used throughout the lecture: translate the English sentence

*“I enjoyed the movie transformers”*

into Tamil:

*“Naan transfarmar padaththai rasiththen”.*

#### 2.1.1 The encoder RNN



- The encoder reads “I”, “enjoyed”, “the”, “movie”, “transformers” one word per time step and produces hidden states  $h_1, \dots, h_5$  ( $h_i$  = hidden state of the encoder RNN after reading word  $i$ ).
- **The final state  $h_5$  is called the concept vector, thought vector, context vector or annotation.** It is meant to summarise the whole sentence.

#### 2.1.2 The decoder RNN

- The decoder’s initial state is the concept vector:  $s_0 = h_5$ .
- At each step it takes the previously produced word as input (“Naan” to produce “transfarmar”, and so on), updates its hidden state  $s_t$ , and outputs a probability distribution over Tamil words.
- $s_i$  = hidden state of the decoder RNN at step  $i$ .

#### 2.1.3 The problem: no alignment

##### The limitation stated in the lecture

The plain seq2seq model **does not care about the alignment of words between the source sentence and the target sentence**. The decoder sees the source only through the single vector  $h_5$ . When it outputs “padaththai” (“movie”), it cannot specifically look at the word “movie” in the source. **The attention mechanism resolves this.**

##### Intuition

Imagine reading a long paragraph once, closing the book, and then being asked to translate it word by word from memory. You would lose details, especially from the beginning. A

human translator instead keeps the source open and glances back at the relevant word while writing each output word. Attention gives the decoder that “glance back” ability.

## 2.2 Attention: a quick tour

### 2.2.1 Step 1: keep all encoder states

Instead of passing only  $h_5$ , make a copy of **all** the encoder hidden states ( $h_1, \dots, h_5$ ) and make them available to the decoder. Once they are available, the encoder block can be dropped (until decoding completes). The attention mechanism consumes these vectors as one of its inputs.

### 2.2.2 Step 2: build a context vector for every output step

At decoder time step  $t$  (while producing output word  $y_t$ ) compute

$$\mathbf{c}_t = \sum_{i=1}^n \alpha_{ti} \mathbf{h}_i \quad : \text{context vector for output } y_t,$$

where  $n$  is the number of source words and  $t$  is the decoder time step.

In matrix form (encoder states as columns):

$$\mathbf{c}_t = \begin{bmatrix} | & | & | & | & | \\ \mathbf{h}_1 & \mathbf{h}_2 & \mathbf{h}_3 & \mathbf{h}_4 & \mathbf{h}_5 \\ | & | & | & | & | \end{bmatrix} \begin{bmatrix} \alpha_{t1} \\ \alpha_{t2} \\ \alpha_{t3} \\ \alpha_{t4} \\ \alpha_{t5} \end{bmatrix}.$$

The lecture walks through this for each decoder step:

| Step $t$ | Output word | Weights used                                   | Context |
|----------|-------------|------------------------------------------------|---------|
| 1        | Naan        | $\alpha_{11}, \alpha_{12}, \dots, \alpha_{15}$ | $c_1$   |
| 2        | transfarmar | $\alpha_{21}, \dots, \alpha_{25}$              | $c_2$   |
| 3        | padaththai  | $\alpha_{31}, \dots, \alpha_{35}$              | $c_3$   |
| 4        | rasiththen  | $\alpha_{41}, \dots, \alpha_{45}$              | $c_4$   |

#### Intuition

$\alpha_{ti}$  answers “while writing output word  $t$ , how much should I look at input word  $i$ ?”. The context vector  $c_t$  is therefore a *custom summary of the source built specifically for this output word*, instead of one fixed summary for all words.

### 2.2.3 Step 3: how the weights are computed

$$\alpha_{ti} = \text{align}(y_t, h_i) = \frac{\exp(\text{score}(s_{t-1}, h_i))}{\sum_{i'=1}^n \exp(\text{score}(s_{t-1}, h_{i'}))}$$

- $s_{t-1}$ : the decoder state just before producing  $y_t$  (“what I have written so far and what I need next”).
- $\text{score}(s_{t-1}, h_i)$ : a raw relevance number (often written  $e_{ti}$ ).
- Softmax over  $i$  turns the  $n$  scores into weights that are positive and sum to 1:  $\sum_{i=1}^n \alpha_{ti} = 1$ .

### 2.2.4 Alignment of words

Arranging all  $\alpha_{ti}$  in a grid (rows = target words, columns = source words) gives an **alignment matrix**. For our example the bright (large) entries are:

|             | I             | enjoyed       | the | film          | transformers  |
|-------------|---------------|---------------|-----|---------------|---------------|
| Naan        | $\alpha_{11}$ |               |     |               |               |
| transfarmar |               |               |     |               | $\alpha_{25}$ |
| padaththai  |               |               |     | $\alpha_{34}$ |               |
| rasiththen  |               | $\alpha_{42}$ |     |               |               |

Naan  $\leftrightarrow$  I, transfarmar  $\leftrightarrow$  transformers, padaththai  $\leftrightarrow$  film/movie, rasiththen  $\leftrightarrow$  enjoyed. Note the alignment is *not* diagonal: Tamil word order differs from English, and attention learns this re-ordering on its own, without being told.

### 2.2.5 What can be parallelised?

**Question:** can  $\alpha_{ti}$  be computed in parallel for all  $i$  at time step  $t$ ?

**Yes.**  $h_i$  is available for all  $i$ , and  $s_{t-1}$  is also available at time step  $t$ . All  $n$  scores are independent of each other, so they can be computed at the same time.

**Take-away:** attention can be parallelised.

**Question:** can  $h_i$ , for all  $i$ , be computed in parallel?

**No!**  $h_i$  needs  $h_{i-1}$ , which needs  $h_{i-2}$ , ..., down to  $h_0$ . The encoder must run  $n$  steps one after another. (The same holds for the decoder states  $s_t$ .)

## 2.3 The major limitation

Everything about the RNN-based seq2seq model seems good so far: with attention, these models performed well for translation. **However, there is a major limitation in training the model: given a training example, we cannot parallelise the sequence of computations across time steps.**

### Intuition

GPUs are fast because they do thousands of independent operations simultaneously. An RNN forces them to wait: step 5 cannot begin until step 4 finishes. For a sentence of 100 words, that is 100 sequential steps per layer, and the GPU sits mostly idle. Longer sentences mean proportionally slower training. Also, gradients must travel back through all these steps, being multiplied at every step, so they can shrink to nothing (**vanishing**) or blow up (**exploding**).

### The Wishlist

Come up with a new architecture that

1. incorporates the attention mechanism,
2. allows parallelisation, and
3. (of course) gets rid of the vanishing/exploding gradient problems.

The answer is the **Transformer**.

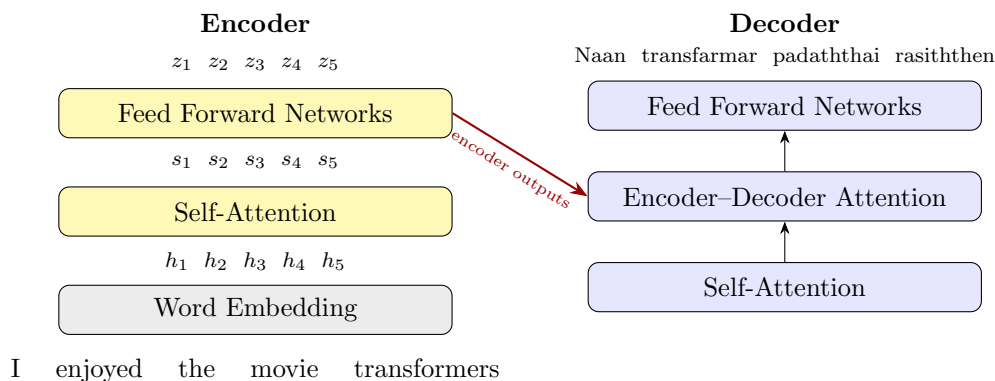
### 3 Module 1.2 — Attention Is All You Need

The Transformer was introduced in the paper “*Attention Is All You Need*” (Vaswani et al., 2017). The title’s message: we do not need RNNs at all. Attention alone (plus simple feed-forward layers) is enough.

#### 3.1 Transition from RNN seq2seq to Transformers

The lecture builds the transition in three pictures:

1. **RNN encoder–decoder:** encoder RNN,  $s_0 = h_5$ , decoder RNN.
2. **With attention:** encoder states  $h_1, \dots, h_5$  feed attention weights  $\alpha_{11}, \dots, \alpha_{15}$  into the decoder.
3. **Transformer:** the recurrence is removed and replaced by attention blocks.



- **Encoder:** Word Embedding  $\rightarrow$  Self-Attention  $\rightarrow$  Feed Forward Networks.
- **Decoder:** Self-Attention  $\rightarrow$  Encoder-Decoder Attention  $\rightarrow$  Feed Forward Networks.
- Each component will be studied one at a time and then connected to form the final architecture. Week 1 covers the encoder side in full.

#### 3.2 Self-attention

##### 3.2.1 Inputs and outputs

- The inputs are vectors (word embeddings) and the outputs are also vectors.
- We already know what attention is: **all it requires is a pair of vectors as input** (one asking, one being scored).
- You can think of the word embeddings as the  $h_i$  of the RNN encoder, if you wish to compare.

##### 3.2.2 Why do we need attention *within* a sentence?

Example 1: “*The animal didn’t cross the street because **it** was too tired.*” Does “it” refer to “animal” or “street”? To “animal” (streets do not get tired).

Example 2: “*The animal didn’t cross the street because **it** was congested.*” Now “it” refers to “street”.

**Intuition**

The *same word* “it” needs a *different meaning* depending on the other words in the sentence. A fixed embedding for “it” cannot do that. So each word must look at the other words of **its own** sentence and update its representation accordingly. That is attention applied to a sequence and itself, hence **self-attention** (distinguished from **cross-attention**, where one sequence looks at a different sequence, seen later).

Therefore it is important to establish a strong connection between “it” and “street” or “animal”, *based on the context*. This calls for an attention mechanism.

**3.2.3 The goal****Definition**

Given a word in a sentence, compute the **relational score** between that word and the rest of the words in the sentence, such that the score is higher if they are related contextually.

The lecture shows the (illustrative) score table:

|        | The  | animal     | didn't | cross | the  | street | because | it   |
|--------|------|------------|--------|-------|------|--------|---------|------|
| The    | 0.6  | 0.1        | 0.05   | 0.05  | 0.02 | 0.02   | 0.02    | 0.1  |
| animal | 0.02 | 0.5        | 0.06   | 0.15  | 0.02 | 0.05   | 0.01    | 0.12 |
| didn't | 0.01 | 0.35       | 0.45   | 0.1   | 0.01 | 0.02   | 0.01    | 0.03 |
| ⋮      |      |            |        |       |      |        |         |      |
| it     | 0.01 | <b>0.6</b> | 0.02   | 0.1   | 0.01 | 0.2    | 0.01    | 0.01 |

In the row for “it”, the largest entry (0.6) is at “animal”. That is exactly the contextual link we wanted the model to discover.

**3.2.4 Rows as  $s_i$ , columns as  $h_j$** 

For convenience, think of the row headers as  $s_i$  and the column headers as  $h_j$  (exactly like decoder states vs. encoder states in seq2seq attention).

**The crucial difference from seq2seq**

In seq2seq,  $h_j$  for all  $j$  were available, but  $s_i$  was obtained *one at a time* (each decoder state waited for the previous one).

In self-attention, **both**  $s_i$  and  $h_j$  for all  $i, j$  are available **all the time** (they are all just the word embeddings of the input).

**Question: does this let us compute all the rows in parallel? Yes.** Nothing in row  $i$  depends on row  $i - 1$ .

**3.3 Choice of attention (score) function****3.3.1 Recall the seq2seq score**

$$\text{score}(s_{t-1}, h_j) = V_a^T \tanh(U_{att} s_{t-1} + W_{att} h_j)$$

Break it down:

- **Three vectors** ( $s, h, v$ ) are involved in computing the score at each decoder time step:  $s_{t-1}$ ,  $h_j$  and the learned vector  $V_a$ .
- **Two linear transformations:**  $U_{att}s_{t-1}$  and  $W_{att}h_j$ .
- **One non-linearity:**  $\tanh$ .
- **Finally a dot product:** with  $V_a$ , giving a single number.

### 3.3.2 The problem

The input to the self-attention module is **only the word embeddings**  $h_j$  for all  $j$ . There is no separate  $s$ . So we need to get **three vectors for each word embedding**. How?

- **By matrix transformation!** Multiply  $h_j$  by a matrix to get a new vector.
- **How many matrices do we need? 3** — one for each role.
- **The values of the elements in these matrices?** They are **learned** (parameters trained by backpropagation), not hand-set.

### 3.4 Transformation matrices: query, key, value

$$\mathbf{q}_j = W_Q \mathbf{h}_j, \quad \mathbf{k}_j = W_K \mathbf{h}_j, \quad \mathbf{v}_j = W_V \mathbf{h}_j$$

- $\mathbf{q}_j$  is called the **query vector** for the word embedding  $h_j$ .
- $\mathbf{k}_j$  is called the **key vector** for the word embedding  $h_j$ .
- $\mathbf{v}_j$  is called the **value vector** for the word embedding  $h_j$ .
- $W_Q$ ,  $W_K$  and  $W_V$  are the respective linear transformation (parameter) matrices. They are **shared across all positions**: the same  $W_Q$  is used for every word.

#### Intuition: the library / search-engine analogy

- **Query** — what a word is *looking for*. “it” asks: “which noun am I referring to?”
- **Key** — what a word *offers as a label*. “animal” advertises: “I am a living noun that could be tired.”
- **Value** — the *content* a word hands over once it is chosen.

Match query against keys (dot product) → decide how much to take from each word → take that much of each value. Keys decide *who* is relevant; values decide *what* gets passed on.

#### Why not just use $h_j$ directly for everything?

If query = key = value =  $h_j$ , then the score of a word with itself,  $h_j^T h_j = \|h_j\|^2$ , tends to dominate, and the relation “ $i$  attends to  $j$ ” would always equal “ $j$  attends to  $i$ ” (symmetric). Language relations are not symmetric (“it” needs “animal” more than “animal” needs “it”). Separate  $W_Q$  and  $W_K$  make the score  $q_i^T k_j = h_i^T W_Q^T W_K h_j$  **asymmetric** and learnable. A separate  $W_V$  lets the model decide what information to send, independently of how matching is done.

### 3.5 Computing the first output $z_1$ , step by step

Focus on computing the first output of the self-attention layer. Toy numbers from the lecture ( $d_k = 2$ ):

| Word                               | query $q_j$    | key $k_j$      | value $v_j$  |
|------------------------------------|----------------|----------------|--------------|
| I ( $h_1 = [1, 0, 0]$ )            | $[-0.1, 0.25]$ | $[0.3, 0.2]$   | $[0.1, 0.5]$ |
| ...                                | ...            | ...            | ...          |
| transformers ( $h_5 = [1, 1, 1]$ ) | $[0.2, 0.7]$   | $[0.11, 0.89]$ | $[0, 0.4]$   |

**Compare with seq2seq:** there the score was  $\text{score}(s_{t-1}, h_j)$  with  $s_{t-1}$  fixed and  $h_j$  variable. Here it is  $\text{score}(q_1, k_j)$  with  $q_1$  fixed and  $k_j$  variable. The score function is now simply a **dot product**.

**Step 1:** Raw scores of word 1 against every word:

$$e_{1j} = [q_1 \cdot k_1, q_1 \cdot k_2, \dots, q_1 \cdot k_5].$$

**Step 2:** Normalise:  $\alpha_{1j} = \text{softmax}(e_{1j})$ .

**Step 3:** Weighted sum of values:

$$z_1 = \sum_{j=1}^5 \alpha_{1j} v_j.$$

**What about  $z_2$ ?** Same procedure with  $q_2$ :  $e_{2j} = [q_2 \cdot k_1, \dots, q_2 \cdot k_5]$ ,  $\alpha_{2j} = \text{softmax}(e_{2j})$ ,  $z_2 = \sum_j \alpha_{2j} v_j$ . **Repeat for all other  $z$ .**

#### Worked example with the lecture's numbers (using only words 1 and 5)

Suppose, to keep arithmetic small, the sentence had only “I” and “transformers”.

$$q_1 \cdot k_1 = (-0.1)(0.3) + (0.25)(0.2) = -0.03 + 0.05 = 0.02$$

$$q_1 \cdot k_5 = (-0.1)(0.11) + (0.25)(0.89) = -0.011 + 0.2225 = 0.2115$$

Softmax of  $[0.02, 0.2115]$ :  $\alpha_{11} \approx 0.4523$ ,  $\alpha_{15} \approx 0.5477$ .

$$z_1 = 0.4523 [0.1, 0.5] + 0.5477 [0, 0.4] \approx [0.0452, 0.4452].$$

“I” ends up looking slightly more at “transformers” than at itself, because  $q_1$  aligns more with  $k_5$ .

#### Key Idea

Computing  $z_2$  does not need  $z_1$ . Every  $z_i$  needs only  $q_i$  and the full set of keys and values, which are available from the start. So all  $z_i$  can be computed **simultaneously**. This is the parallelism the RNN lacked.

### 3.6 Vectorising: all outputs in one go

“Wait, can we vectorise all these computations and compute  $z_1, z_2, \dots, z_T$  in one go?” Yes. Stack the embeddings as columns:

$$H = [h_1 \ h_2 \ \dots \ h_T] \in \mathbb{R}^{d_{\text{model}} \times T}.$$

Then

$$\begin{aligned} Q &= [q_1 \ q_2 \ \cdots \ q_T] = W_Q H, & Q &\in \mathbb{R}^{64 \times T}, \\ K &= [k_1 \ k_2 \ \cdots \ k_T] = W_K H, & K &\in \mathbb{R}^{64 \times T}, \\ V &= [v_1 \ v_2 \ \cdots \ v_T] = W_V H, & V &\in \mathbb{R}^{64 \times T}. \end{aligned}$$

So  $W_Q, W_K, W_V \in \mathbb{R}^{64 \times 512}$  (they map 512-dim embeddings to 64-dim vectors).

### 3.6.1 The vectorised output: scaled dot-product attention

$$Z = [z_1, z_2, \dots, z_T] = \text{softmax}\left(\frac{Q^T K}{\sqrt{d_k}}\right) V^T$$

where  $d_k$  is the dimension of the key vector ( $d_k = 64$ ).

**Shape check (do this every time):**

$$\begin{aligned} \dim(Q^T K) &: (T \times 64)(64 \times T) = T \times T, \\ \dim(Z) &: (T \times T)(T \times 64) = T \times 64. \end{aligned}$$

Row  $i$  of  $Q^T K$  holds  $[q_i \cdot k_1, \dots, q_i \cdot k_T]$ : that is exactly  $e_{ij}$  from before, for all  $i$  at once. Softmax is applied **row-wise**, so each row becomes one word's attention distribution. Multiplying by  $V^T$  forms every weighted sum at once; row  $i$  of  $Z$  is  $z_i$ .

#### Convention note

The lecture writes embeddings as *column* vectors, giving  $\text{softmax}(Q^T K / \sqrt{d_k}) V^T$ . The original paper writes  $h_j$  as a *row* vector, so it writes

$$\text{softmax}\left(\frac{Q K^T}{\sqrt{d_k}}\right) V.$$

Same computation, transposed bookkeeping. In an exam, match the convention the question uses.

### 3.6.2 Why divide by $\sqrt{d_k}$ ? (“scaled” dot product)

Since  $\sqrt{d_k}$  scales the values of  $Q^T K$ , this is called the **scaled dot product**.

**Derivation.** Suppose the components of  $q$  and  $k$  are independent with mean 0 and variance 1. Then each product  $q_i k_i$  has mean 0 and variance  $\mathbb{E}[q_i^2] \mathbb{E}[k_i^2] = 1$ , so

$$q \cdot k = \sum_{i=1}^{d_k} q_i k_i \quad \Rightarrow \quad \mathbb{E}[q \cdot k] = 0, \quad \text{Var}(q \cdot k) = d_k.$$

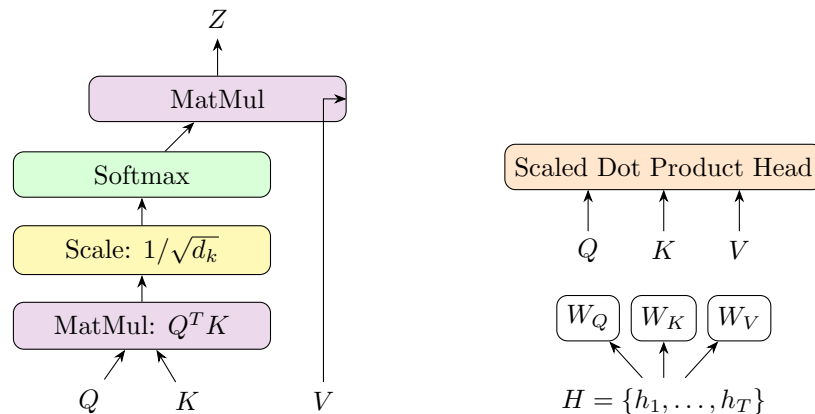
With  $d_k = 64$  the standard deviation of scores is  $\sqrt{64} = 8$ . Scores of size  $\pm 16$  or  $\pm 24$  are common, which makes softmax nearly one-hot (saturated) and its gradients nearly zero. Dividing by  $\sqrt{d_k}$  gives

$$\text{Var}\left(\frac{q \cdot k}{\sqrt{d_k}}\right) = \frac{d_k}{d_k} = 1,$$

keeping softmax in a healthy range regardless of  $d_k$ .

**Intuition**

Adding more dimensions adds more random terms to the dot product, so it naturally grows. Scaling by  $\sqrt{d_k}$  removes this “bigger just because the vector is longer” effect, so scores reflect genuine similarity rather than dimension size.

**3.6.3 The computational graph of one head**

1. MatMul:  $Q^T K$  (all pairwise similarities).
2. Scale by  $1/\sqrt{d_k}$ .
3. Softmax (row-wise).
4. MatMul with  $V^T$ .

The whole box (with its own  $W_Q, W_K, W_V$  acting on  $H$ ) is called one **scaled dot-product attention head**.

**3.7 Multi-head attention****3.7.1 Two-head attention**

Take two heads side by side. Each has its *own*  $W_Q, W_K, W_V$  but reads the *same* input  $H = \{h_1, \dots, h_T\}$ .

**3.7.2 Motivation (analogy with CNN filters)****Intuition**

**What is the significance of having more than one filter/kernel in a CNN layer?**  
To learn more abstract representations and capture more meaningful interactions between inputs: one filter detects edges, another textures, and so on.

Similarly, we can have more than one self-attention head with different parameter matrices ( $W_{Q_i}, W_{K_i}, W_{V_i}$ ), hoping each learns subtle contextual information. This motivates **multi-head attention**, a simple extension of single-head attention.

Just as each kernel independently learns its feature in a CNN, each head independently computes attention in Transformers. **Parallel computation!**

### 3.7.3 Empirical evidence (Tensor2Tensor visualisation)

- **Single head:** “it” is strongly connected to “was”.
- **Two heads:** in the first head, “it” is strongly connected to “was”; in the *second* head, “it” is strongly connected to “animal”.
- So it is evident (empirically) that adding more than one attention head helps capture **different contextual information** of the sentence.

#### Intuition

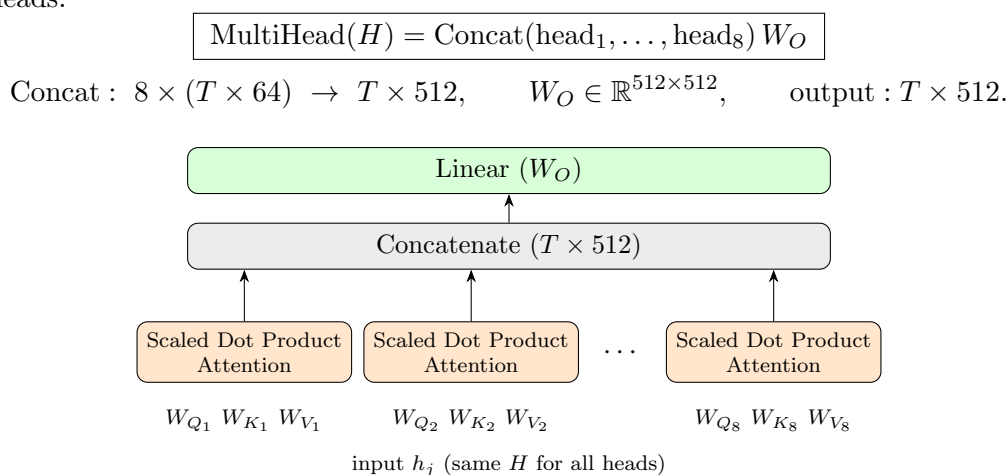
A single softmax row must split one unit of attention among all words. If “it” needs to track both its verb (“was”, grammar) and its referent (“animal”, meaning), a single head is forced to compromise. Several heads give several independent “spotlights”.

### 3.7.4 Assembling multiple heads

Recall how  $Q, K, V$  are obtained from  $H$ . To extend this to multi-head attention:

1. Head  $i$  uses its own matrices:  $Q_i = W_{Q_i}H$ ,  $K_i = W_{K_i}H$ ,  $V_i = W_{V_i}H$ .
2. Each head computes  $\text{head}_i = \text{softmax}(Q_i^T K_i / \sqrt{d_k}) V_i^T \in \mathbb{R}^{T \times 64}$ .
3. **Concatenate** the head outputs side by side.
4. Pass the concatenation through a final **Linear** layer  $W_O$ .

With 8 heads:



#### Why $d_k = 64$ ?

$d_{\text{model}} = 512$  is split across  $h = 8$  heads:  $d_k = d_v = d_{\text{model}}/h = 512/8 = 64$ . Concatenating 8 outputs of width 64 gives back width 512. So multi-head attention costs about the same as one big 512-dimensional head, but produces 8 different attention patterns.

## 3.8 The encoder layer

### 3.8.1 Terminology

A slight change in terminology. Multi-head attention and the feed-forward network together form an **encoder layer**. The encoder is composed of a stack of  $N = 6$  layers. Therefore each

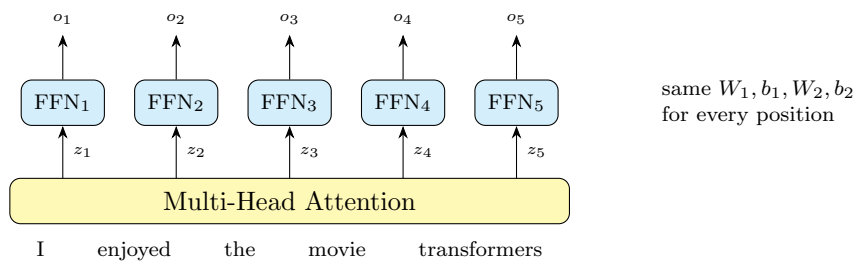
layer in the encoder is composed of **two sub-layers**: multi-head attention and feed-forward neural network. (In the earlier picture the block was labelled “Self-Attention”; it is now “Multi-Head Attention”).)

### 3.8.2 Position-wise feed-forward network

Let  $z_i \in \mathbb{R}^{512}$  be the multi-head attention output at position  $i$ . Each  $z_i$  passes through a small two-layer network:

$$\text{FFN}(z) = \max(0, W_1 z + b_1) W_2 + b_2$$

- Input:  $z_i \in \mathbb{R}^{512}$ .
- Hidden layer width:  $l = 2048$ , with ReLU  $\max(0, \cdot)$ .
- Output:  $o_i \in \mathbb{R}^{512}$ .
- **Identical network for each position**  $z_i$ :  $\text{FFN}_1, \text{FFN}_2, \dots, \text{FFN}_5$  in the picture all have the same weights.



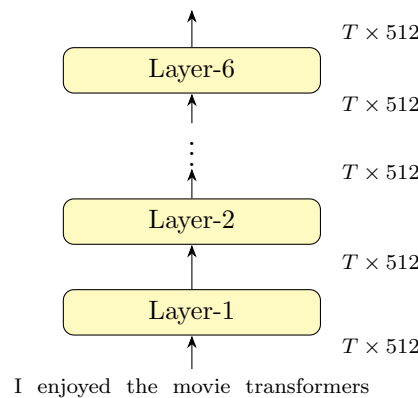
#### Intuition

Attention *mixes* information across words (“communication”). The FFN then *processes* each word’s mixed vector on its own (“computation”), adding non-linearity so the model can represent complex functions. Expanding to 2048 and back to 512 gives a wide hidden space for this processing. Because the FFN treats positions independently, all positions are processed in parallel too.

#### Be Careful

Attention is the **only** place where positions interact. The FFN never looks at other positions. Without the ReLU, stacking layers would collapse into one linear map; the ReLU is the key non-linearity of the layer.

### 3.9 The encoder stack



- The encoder is composed of  $N$  identical layers, and each layer is composed of 2 sub-layers. Every layer takes  $T \times 512$  and outputs  $T \times 512$ , which is why they can be stacked.
- “Identical” means same *structure*; each layer has its *own* weights.
- **Number of parameters:**  $N \times 3 \times 10^6 = 6 \times 3 \text{ million} \approx \mathbf{18 \text{ million parameters}}$ .
- **The computation is parallelised in the horizontal direction** (i.e. across positions, within a training sample) of the encoder stack, **not along the vertical direction** (Layer-2 must wait for Layer-1).
- Denote the output sequence of vectors from the encoder as  $e_j$ , for  $j = 1, \dots, T$  (one output vector per input word). These are what the decoder will attend to.

#### 3.9.1 Where does “3 million per layer” come from?

| Component                                                        | Weights only     | With biases      |
|------------------------------------------------------------------|------------------|------------------|
| $W_Q, W_K, W_V$ for 8 heads: $8 \times 3 \times (64 \times 512)$ | 786,432          | + 1,536          |
| $W_O$ : $512 \times 512$                                         | 262,144          | + 512            |
| <b>Multi-head attention total</b>                                | <b>1,048,576</b> | <b>1,050,624</b> |
| $W_1$ : $512 \times 2048$                                        | 1,048,576        | + 2,048          |
| $W_2$ : $2048 \times 512$                                        | 1,048,576        | + 512            |
| <b>FFN total</b>                                                 | <b>2,097,152</b> | <b>2,099,712</b> |
| <b>Per layer</b>                                                 | <b>3,145,728</b> | <b>3,150,336</b> |
| <b>6 layers</b>                                                  | 18,874,368       | 18,902,016       |

So  $\approx 3\text{M}$  per layer and  $\approx 18\text{--}19\text{M}$  for the encoder, matching the lecture. Notice the FFN holds **two-thirds** of the parameters of each layer.

#### Horizontal vs. vertical

Think of a grid: columns are words, rows are layers. Within a row, all columns are computed at once (horizontal parallelism). Rows are computed bottom to top in order (vertical sequence). The number of sequential steps is the number of layers (6), *not* the number of words. For an RNN it was the number of words.

### 3.10 Cross-attention (encoder–decoder attention): preview

The lecture title includes cross-attention. The encoder outputs  $e_1, \dots, e_T$  feed the decoder's **Encoder–Decoder Attention** block:

- **Queries** come from the decoder (the target words produced so far).
- **Keys and values** come from the encoder outputs  $e_j$ .
- The formula is the same scaled dot-product attention; only the sources of  $Q$  versus  $K, V$  differ.
- If the decoder has  $m$  positions and the encoder  $T$ , the attention matrix is  $m \times T$  (not square).

#### Self-attention vs. cross-attention

**Self-attention:**  $Q, K, V$  all come from the *same* sequence.

**Cross-attention:**  $Q$  from one sequence (decoder),  $K$  and  $V$  from another (encoder). This is the Transformer's version of the seq2seq alignment  $\alpha_{ti}$  from Module 1.1.

## 4 Big-Picture Comparison and Formula Sheet

### 4.1 RNN seq2seq with attention vs. Transformer encoder

| Aspect                                  | RNN seq2seq + attention                                  | Transformer encoder            |
|-----------------------------------------|----------------------------------------------------------|--------------------------------|
| How word vectors are built              | Sequentially, $h_t = f(h_{t-1}, x_t)$                    | All at once by self-attention  |
| Parallel across positions?              | No (encoder states); yes (attention weights at one step) | Yes, everything within a layer |
| Sequential steps                        | $T$ (sentence length)                                    | $N$ (number of layers, 6)      |
| Score function                          | $V_a^T \tanh(Us + Wh)$                                   | $q^T k / \sqrt{d_k}$           |
| Path between distant words              | Up to $T$ steps                                          | 1 step (direct attention)      |
| Vanishing/exploding gradients over time | Yes                                                      | No recurrence through time     |
| Contextual views                        | One                                                      | Several (multi-head)           |
| Cost of attention matrix                | —                                                        | $O(T^2 d_k)$ per head          |

### 4.2 Formula sheet

|                          |                                                                                                        |
|--------------------------|--------------------------------------------------------------------------------------------------------|
| Context vector           | $c_t = \sum_{i=1}^n \alpha_{ti} h_i$                                                                   |
| Attention weights        | $\alpha_{ti} = \frac{\exp(\text{score}(s_{t-1}, h_i))}{\sum_{i'} \exp(\text{score}(s_{t-1}, h_{i'}))}$ |
| Additive (seq2seq) score | $V_a^T \tanh(U_{att} s_{t-1} + W_{att} h_j)$                                                           |
| Query / key / value      | $q_j = W_Q h_j, k_j = W_K h_j, v_j = W_V h_j$                                                          |
| Single output            | $e_{ij} = q_i \cdot k_j, \alpha_{ij} = \text{softmax}_j(e_{ij}), z_i = \sum_j \alpha_{ij} v_j$         |
| Vectorised               | $Q = W_Q H, K = W_K H, V = W_V H$                                                                      |
| Scaled dot-product       | $Z = \text{softmax}(Q^T K / \sqrt{d_k}) V^T$ (paper: $\text{softmax}(Q K^T / \sqrt{d_k}) V$ )          |
| Multi-head               | $\text{Concat}(\text{head}_1, \dots, \text{head}_h) W_O$                                               |
| FFN                      | $\max(0, W_1 z + b_1) W_2 + b_2$                                                                       |
| Head size                | $d_k = d_v = d_{\text{model}} / h = 512 / 8 = 64$                                                      |
| Encoder parameters       | $\approx N \times 3 \times 10^6 = 18 \times 10^6$                                                      |

**4.3 Shape cheat-sheet** ( $d_{\text{model}} = 512$ ,  $h = 8$ ,  $d_k = d_v = 64$ ,  $d_{ff} = 2048$ )

|                            |                                     |
|----------------------------|-------------------------------------|
| $H$                        | $512 \times T$                      |
| $W_Q, W_K, W_V$ (per head) | $64 \times 512$                     |
| $Q, K, V$ (per head)       | $64 \times T$                       |
| $Q^T K$                    | $T \times T$                        |
| head output $Z$            | $T \times 64$                       |
| Concat of 8 heads          | $T \times 512$                      |
| $W_O$                      | $512 \times 512$                    |
| $W_1 / W_2$                | $512 \times 2048 / 2048 \times 512$ |
| Layer output               | $T \times 512$                      |

## Part II — Question Bank

### 5 Numerical Problems (with full worked solutions)

Useful values:  $e \approx 2.7183$ ,  $e^2 \approx 7.389$ ,  $e^3 \approx 20.086$ ,  $e^{0.7071} \approx 2.0281$ ,  $e^{1.4142} \approx 4.1132$ ,  $\sqrt{2} \approx 1.4142$ .

#### A. Softmax and attention weights

##### N1. Basic softmax

Scores for three source words are  $e = [2, 1, 0]$ . Find the attention weights.

$e^2 = 7.389$ ,  $e^1 = 2.718$ ,  $e^0 = 1$ ; sum = 11.107.

$\alpha = [7.389, 2.718, 1]/11.107 = [0.6652, 0.2447, 0.0900]$ . They sum to 1.

##### N2. Softmax with logs

Compute  $\text{softmax}([0, \ln 2, \ln 3])$  exactly.

Exponentials are 1, 2, 3; sum 6. Answer  $[1/6, 1/3, 1/2]$ .

##### N3. Shift invariance

Compute  $\text{softmax}([3, 4, 5])$  and  $\text{softmax}([0, 1, 2])$ . What do you notice?

Both equal  $[0.0900, 0.2447, 0.6652]$ . The second is the first minus 3; softmax is unchanged by adding/subtracting a constant because  $e^c$  cancels. (Implementations subtract the max score for numerical stability using exactly this.)

##### N4. Equal scores

A query produces equal scores  $[2, 2, 2, 2]$  with four keys. What are the weights, and what is the output  $z$  in terms of the values?

$\alpha = [0.25, 0.25, 0.25, 0.25]$ .  $z = \frac{1}{4}(v_1 + v_2 + v_3 + v_4)$ : the plain average of the values. Equal scores mean “no preference”, so attention reduces to averaging.

##### N5. Effect of scaling (sharpness)

Raw scores  $[4, 2, 0]$  with  $d_k = 4$ . Compute softmax before and after dividing by  $\sqrt{d_k}$ .

Unscaled:  $\text{softmax}([4, 2, 0]) = [0.8668, 0.1173, 0.0159]$ .

Scaled:  $[4, 2, 0]/2 = [2, 1, 0] \Rightarrow [0.6652, 0.2447, 0.0900]$ .

Scaling makes the distribution flatter; the top weight drops from 0.87 to 0.67.

**N6. Saturation and gradients**

For two scores  $[x, 0]$ , the top weight is  $p = \sigma(x)$  and  $\partial p / \partial x = p(1 - p)$ . Compare  $x = 8$  with  $x = 1$ .

$x = 8$ :  $p = 0.99966$ , gradient = 0.00034.

$x = 1$ :  $p = 0.7311$ , gradient =  $0.7311 \times 0.2689 = 0.1966$ .

The gradient is about  $580\times$  smaller when saturated. Large unscaled dot products cause exactly this, which is why we divide by  $\sqrt{d_k}$ .

**N7. Variance of dot products**

Components of  $q, k \in \mathbb{R}^{d_k}$  are i.i.d. with mean 0, variance 1. (a)  $d_k = 16$ : variance and std of  $q \cdot k$ ? (b) Same for  $d_k = 64$ . (c) After dividing by  $\sqrt{d_k}$ ?

$\text{Var}(q \cdot k) = d_k$ . (a) variance 16, std 4. (b) variance 64, std 8. (c)  $\text{Var} = d_k/d_k = 1$ , std 1 in both cases.

**B. Seq2seq attention****N8. Context vector**

$h_1 = [1, 0]$ ,  $h_2 = [2, 1]$ ,  $h_3 = [0, 3]$  and  $\alpha_t = [0.1, 0.6, 0.3]$ . Find  $c_t$ .

$$c_t = 0.1[1, 0] + 0.6[2, 1] + 0.3[0, 3] = [0.1 + 1.2 + 0, 0 + 0.6 + 0.9] = [1.3, 1.5].$$

**N9. Full seq2seq attention step**

Four source words have  $h_1 = [1, 0]$ ,  $h_2 = [0, 1]$ ,  $h_3 = [1, 1]$ ,  $h_4 = [2, 0]$  and scores  $e_t = [1, 3, 0, 2]$ . Find  $\alpha_t$  and  $c_t$ . Which source word is aligned to  $y_t$ ?

$$e^1 = 2.718, e^3 = 20.086, e^0 = 1, e^2 = 7.389; \text{sum} = 31.193.$$

$$\alpha_t = [0.0871, 0.6439, 0.0321, 0.2369].$$

$$c_t = 0.0871[1, 0] + 0.6439[0, 1] + 0.0321[1, 1] + 0.2369[2, 0] = [0.593, 0.676].$$

Largest weight is on word 2, so  $y_t$  is aligned to source word 2.

**N10. Additive (Bahdanau) score**

$U_{att} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ ,  $W_{att} = \begin{bmatrix} 1 & 1 \\ 0 & -1 \end{bmatrix}$ ,  $V_a = [1, 2]^T$ ,  $s_{t-1} = [1, 0]^T$ ,  $h_j = [0, 1]^T$ . Compute  $\text{score}(s_{t-1}, h_j)$ .

$$U_{att}s = [1, 0], W_{att}h = [0 + 1, 0 - 1] = [1, -1]. \text{Sum} = [2, -1].$$

$$\tanh([2, -1]) = [0.9640, -0.7616].$$

$$V_a^T(\cdot) = 1(0.9640) + 2(-0.7616) = [-0.5592].$$

Note the ingredients: 3 vectors ( $s, h, V_a$ ), 2 linear maps, 1 non-linearity, 1 dot product.

**N11. Reading an alignment matrix**

Rows are target words, columns source words (I, enjoyed, the, film, transformers):

|             |      |      |      |      |      |
|-------------|------|------|------|------|------|
| Naan        | 0.80 | 0.05 | 0.05 | 0.05 | 0.05 |
| transfarmar | 0.02 | 0.03 | 0.05 | 0.10 | 0.80 |
| padaththai  | 0.02 | 0.03 | 0.15 | 0.75 | 0.05 |
| rasiththen  | 0.05 | 0.85 | 0.03 | 0.02 | 0.05 |

Give the alignment and say whether it is diagonal.

Naan→I, transfarmar→transformers, padaththai→film, rasiththen→enjoyed (i.e.  $\alpha_{11}, \alpha_{25}, \alpha_{34}, \alpha_{42}$  are largest). Not diagonal: Tamil puts the verb last, and attention learns the re-ordering. Each row sums to 1.

**N12. Counting attention weights**

- (a) Seq2seq: source 5 words, target 4 words. How many  $\alpha_{ti}$ ?
- (b) Self-attention over  $T = 8$  words, one head?
- (c) Same with 8 heads?
- (d) Cross-attention, decoder 4 positions, encoder 5 positions?

(a)  $4 \times 5 = 20$ . (b)  $8 \times 8 = 64$ . (c)  $8 \times 64 = 512$ . (d)  $4 \times 5 = 20$  (matrix is  $4 \times 5$ , not square).

**C. Self-attention computations****N13. Lecture numbers**

From the lecture:  $q_1 = [-0.1, 0.25]$ ,  $k_1 = [0.3, 0.2]$ ,  $v_1 = [0.1, 0.5]$ ,  $q_5 = [0.2, 0.7]$ ,  $k_5 = [0.11, 0.89]$ ,  $v_5 = [0, 0.4]$ . (a) Compute  $q_1 \cdot k_1$ ,  $q_1 \cdot k_5$ ,  $q_5 \cdot k_1$ ,  $q_5 \cdot k_5$ . (b) Treating the sentence as only these two words (no scaling), compute  $z_1$  and  $z_5$ .

- (a)  $q_1 \cdot k_1 = -0.03 + 0.05 = 0.02$ ;  $q_1 \cdot k_5 = -0.011 + 0.2225 = 0.2115$ ;  
 $q_5 \cdot k_1 = 0.06 + 0.14 = 0.20$ ;  $q_5 \cdot k_5 = 0.022 + 0.623 = 0.645$ .
- (b)  $\alpha_1 = \text{softmax}([0.02, 0.2115]) = [0.4523, 0.5477]$ ,  $z_1 = [0.0452, 0.4452]$ .  
 $\alpha_5 = \text{softmax}([0.20, 0.645]) = [0.3906, 0.6094]$ ,  $z_5 = [0.0391, 0.4391]$ .  
 Observation:  $q \cdot k$  is not symmetric in general:  $q_1 \cdot k_5 = 0.2115 \neq q_5 \cdot k_1 = 0.20$ .

**N14. Single output, unscaled**

$q_1 = [1, 0]$ ; keys  $k_1 = [1, 1]$ ,  $k_2 = [0, 1]$ ,  $k_3 = [1, 0]$ ; values  $v_1 = [1, 0]$ ,  $v_2 = [0, 1]$ ,  $v_3 = [1, 1]$ . Compute  $z_1$  with no scaling.

Scores  $[1, 0, 1]$ .  $\alpha = [e, 1, e]/(2e + 1) = [0.4223, 0.1554, 0.4223]$ .  
 $z_1 = 0.4223[1, 0] + 0.1554[0, 1] + 0.4223[1, 1] = [0.8446, 0.5777]$ .

**N15. Same with scaling**

Repeat N14 with scaling,  $d_k = 2$ .

Scaled scores  $[0.7071, 0, 0.7071]$ ;  $e^{0.7071} = 2.0281$ ;  $\text{sum} = 5.0562$ .

$\alpha = [0.4011, 0.1978, 0.4011]$ ;  $z_1 = [0.8022, 0.5989]$ . Flatter weights than N14.

### N16. Computing $q, k, v$ from embeddings

$W_Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ ,  $W_K = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 0 \end{bmatrix}$ ,  $W_V = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$ . Embeddings (lecture): I =  $[1, 0, 0]$ , enjoyed =  $[0, 1, 0]$ , the =  $[1, 1, 0]$ . (a) Compute  $q, k, v$  for each word. (b) Shape of  $W_Q$  and what it maps between.

| Word    | $q = W_Q h$ | $k = W_K h$ | $v = W_V h$ |
|---------|-------------|-------------|-------------|
| (a) I   | $[1, 0]$    | $[0, 1]$    | $[1, 0]$    |
| enjoyed | $[0, 1]$    | $[1, 0]$    | $[0, 1]$    |
| the     | $[1, 1]$    | $[1, 1]$    | $[1, 1]$    |

(b)  $W_Q \in \mathbb{R}^{2 \times 3}$ : maps 3-dim embeddings to 2-dim queries ( $d_k = 2$ ).

### N17. Full self-attention for N16 (matrix form)

Using N16, compute  $Q^T K$ , the scaled attention matrix, and  $Z = \text{softmax}(Q^T K / \sqrt{2}) V^T$ .

$$Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}, K = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}, V = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}.$$

$$Q^T K = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 2 \end{bmatrix}, \quad \frac{Q^T K}{\sqrt{2}} = \begin{bmatrix} 0 & .7071 & .7071 \\ .7071 & 0 & .7071 \\ .7071 & .7071 & 1.4142 \end{bmatrix}.$$

Row-wise softmax: row 1:  $[1, 2.0281, 2.0281] / 5.0562 = [0.1978, 0.4011, 0.4011]$ ; row 2:  $[0.4011, 0.1978, 0.4011]$ ; row 3:  $[2.0281, 2.0281, 4.1132] / 8.1694 = [0.2483, 0.2483, 0.5035]$ .

$$V^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \end{bmatrix}, \text{ so}$$

$$Z = \begin{bmatrix} 0.5989 & 0.8022 \\ 0.8022 & 0.5989 \\ 0.7517 & 0.7517 \end{bmatrix}.$$

Check:  $z_1 = 0.1978[1, 0] + 0.4011[0, 1] + 0.4011[1, 1] = [0.5989, 0.8022]$ . ✓

Note “I” attends least to itself here: in self-attention a word does not automatically attend most to itself.

### N18. Another full matrix

$Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{bmatrix}$ ,  $K = \begin{bmatrix} 0 & 1 & 1 \\ 1 & 0 & 1 \end{bmatrix}$ ,  $V = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$ ,  $d_k = 2$ . Compute  $Z$ .

Attention matrix is the same as N17.  $V^T = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}$ .

$$Z = \begin{bmatrix} 0.1978 + 0.4011 & 0.4011 \\ 0.4011 + 0.4011 & 0.1978 \\ 0.2483 + 0.5035 & 0.2483 \end{bmatrix} = \begin{bmatrix} 0.5989 & 0.4011 \\ 0.8022 & 0.1978 \\ 0.7517 & 0.2483 \end{bmatrix}.$$

### N19. No projections ( $q = k = v = h$ )

If we skipped  $W_Q, W_K, W_V$  and used  $h$  directly, with  $h_1 = [1, 0], h_2 = [0, 1], h_3 = [1, 1]$  (no scaling), compute the attention matrix and  $Z$ . Is the matrix of scores symmetric?

Scores  $HH^T = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 2 \end{bmatrix}$  — **symmetric**. Softmax rows:  $[0.4223, 0.1554, 0.4223],$

$$[0.1554, 0.4223, 0.4223], [0.2119, 0.2119, 0.5761]. Z = \begin{bmatrix} 0.8446 & 0.5777 \\ 0.5777 & 0.8446 \\ 0.7881 & 0.7881 \end{bmatrix}.$$

Symmetric raw scores and no learnable parameters: this is why separate learned  $W_Q, W_K, W_V$  are used.

### N20. Cross-attention (preview)

Two decoder queries  $q_1 = [1, 0], q_2 = [0, 2]$ ; three encoder keys  $k_1 = [1, 0], k_2 = [0, 1], k_3 = [1, 1]$ , values  $v_1 = [2, 0], v_2 = [0, 2], v_3 = [1, 1]$ . No scaling. Compute the attention matrix and outputs.

Scores:  $\begin{bmatrix} 1 & 0 & 1 \\ 0 & 2 & 2 \end{bmatrix}$  (shape  $2 \times 3$ ).

Row 1:  $[0.4223, 0.1554, 0.4223]$ ; row 2:  $[1, e^2, e^2]/(1 + 2e^2) = [0.0634, 0.4683, 0.4683]$ .

$z_1 = 0.4223[2, 0] + 0.1554[0, 2] + 0.4223[1, 1] = [1.2670, 0.7330]$ .

$z_2 = 0.0634[2, 0] + 0.4683[0, 2] + 0.4683[1, 1] = [0.5951, 1.4049]$ .

## D. Multi-head attention and FFN

### N21. Concatenate and project

Two heads for  $T = 2$  positions (rows = positions):  $\text{head}_1 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$ ,  $\text{head}_2 = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}$ ,

$$W_O = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix}. \text{ Compute } \text{Concat}(\text{head}_1, \text{head}_2)W_O.$$

$\text{Concat} = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 0 & 1 & 1 & 1 \end{bmatrix}$  ( $T \times 4$ ).

Row 1:  $1 \cdot [1, 0, 0, 1] + 0 \cdot [1, 1, 0, 0] + 1 \cdot [0, 0, 1, 1] = [3, 2, 1, 2]$ .

Row 2:  $0 + 1 \cdot [0, 1, 1, 0] + 1 \cdot [1, 1, 0, 0] + 1 \cdot [0, 0, 1, 1] = [1, 2, 2, 1]$ .

Output =  $\begin{bmatrix} 3 & 2 & 1 & 2 \\ 1 & 2 & 2 & 1 \end{bmatrix}$ .  $W_O$  mixes the heads' outputs.

### N22. FFN forward pass

$W_1 = \begin{bmatrix} 1 & -1 \\ 2 & 0 \\ -1 & 1 \end{bmatrix}$ ,  $b_1 = [0, -1, 0.5]^T$ ,  $W_2 = \begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & -1 \end{bmatrix}$ ,  $b_2 = [0.5, 0]^T$ ,  $z = [1, 2]^T$ . Compute  $W_2 \max(0, W_1 z + b_1) + b_2$ .

$W_1 z = [1 - 2, 2, -1 + 2] = [-1, 2, 1]$ ;  $+b_1 = [-1, 1, 1.5]$ .

ReLU:  $[0, 1, 1.5]$  (the negative unit is switched off).

$W_2[0, 1, 1.5] = [0 + 0 + 3, 0 + 1 - 1.5] = [3, -0.5]$ ;  $+b_2 = \boxed{[3.5, -0.5]}$ .

Shapes:  $2 \rightarrow 3 \rightarrow 2$ , a mini version of  $512 \rightarrow 2048 \rightarrow 512$ .

### N23. Head dimension

Find  $d_k$  for: (a)  $d_{\text{model}} = 512, h = 8$ ; (b)  $d_{\text{model}} = 512, h = 16$ ; (c)  $d_{\text{model}} = 768, h = 12$ ; (d)  $d_{\text{model}} = 1024, h = 16$ .

(a) 64    (b) 32    (c) 64    (d) 64.

## E. Shapes, parameters, cost

### N24. Shape tracking

$d_{\text{model}} = 512$ ,  $h = 8$ ,  $T = 20$ . Give the shape of:  $H$ ,  $W_Q^{(i)}$ ,  $Q^{(i)}$ ,  $Q^{(i)T}K^{(i)}$ , head output, concatenation,  $W_O$ , layer output, FFN hidden activation for one position.

$H : 512 \times 20$ ;  $W_Q^{(i)} : 64 \times 512$ ;  $Q^{(i)} : 64 \times 20$ ;  $Q^{(i)T}K^{(i)} : 20 \times 20$ ; head output  $20 \times 64$ ; concat  $20 \times 512$ ;  $W_O : 512 \times 512$ ; layer output  $20 \times 512$ ; FFN hidden 2048.

### N25. Parameters of the original encoder layer

$d_{\text{model}} = 512$ ,  $h = 8$ ,  $d_{ff} = 2048$ . Count parameters (a) without biases, (b) with biases, (c) for 6 layers.

(a) Attention:  $8 \cdot 3 \cdot 64 \cdot 512 + 512^2 = 786,432 + 262,144 = 1,048,576$ ; FFN:  $2 \cdot 512 \cdot 2048 = 2,097,152$ ; total 3,145,728.

(b) Biases add 1,536 + 512 (attention) and 2,048 + 512 (FFN): total 3,150,336.

(c)  $6 \times 3,150,336 = 18,902,016 \approx 18\text{M}$ .

### N26. A shortcut formula

Show that, ignoring biases, one layer has  $4d_{\text{model}}^2 + 2d_{\text{model}}d_{ff}$  parameters. Evaluate for  $d_{\text{model}} = 1024, d_{ff} = 4096$ .

$W_Q, W_K, W_V$  across  $h$  heads:  $3 \cdot h \cdot (d_{\text{model}}/h) \cdot d_{\text{model}} = 3d_{\text{model}}^2$ .  $W_O$ :  $d_{\text{model}}^2$ . FFN:  $2d_{\text{model}}d_{ff}$ .  
 Total  $4d_{\text{model}}^2 + 2d_{\text{model}}d_{ff}$ . (Note: independent of  $h$ !)  
 $4(1024)^2 + 2(1024)(4096) = 4,194,304 + 8,388,608 = \boxed{12,582,912}$ .

### N27. A bigger encoder

$d_{\text{model}} = 768$ ,  $h = 12$ ,  $d_{ff} = 3072$ , 12 layers, with biases. Parameters per layer and total?

Attention:  $3 \cdot 768^2 + 768^2 + (3 \cdot 768 + 768) = 2,359,296 + 3,072 = 2,362,368$ .  
 FFN:  $2 \cdot 768 \cdot 3072 + 3072 + 768 = 4,718,592 + 3,840 = 4,722,432$ .  
 Per layer 7,084,800; 12 layers  $\boxed{85,017,600} \approx 85\text{M}$ .

### N28. Multiplications in one head

$T = 10$ ,  $d_k = d_v = 64$ . Count scalar multiplications for (a)  $Q^T K$ , (b) (weights)  $\times V^T$ .

(a)  $T^2$  dot products of length  $d_k$ :  $100 \times 64 = 6,400$ . (b)  $T \times T \times d_v = 6,400$ . Total 12,800.  
 Both grow as  $T^2$ : doubling  $T$  quadruples the cost.

### N29. Memory for attention matrices

$T = 1024$ , 8 heads, 4 bytes per number. Memory for one layer's attention weights? For 6 layers?

$1024^2 \times 8 \times 4 = 33,554,432$  bytes = 32 MiB per layer; 192 MiB for 6 layers.

### N30. Sequential steps

Sentence length  $T = 50$ . How many sequential steps does (a) a 1-layer RNN encoder need, (b) a 6-layer Transformer encoder need (counting one step per layer)? What if  $T = 500$ ?

(a) 50 (and 500 for  $T = 500$ ). (b) 6 in both cases. The Transformer's sequential depth depends on layers, not sentence length.

### N31. Path length

In an RNN, how many steps must information travel from word 1 to word 40? In one self-attention layer?

RNN: 39 recurrent steps (gradient multiplied 39 times, risk of vanishing). Self-attention: 1 (word 40 attends to word 1 directly).

## 6 Subjective / Descriptive Questions

**S1**

What is the concept (thought) vector? Why is relying on it alone a problem?

It is the final encoder hidden state ( $h_5$  in the example), also called thought vector, context vector or annotation. It is the decoder's only view of the source and becomes  $s_0$ . Problems: all information must be squeezed into one fixed-size vector (bottleneck), early words fade for long sentences, and the decoder cannot focus on the specific source word relevant to the current output word (no alignment).

**S2**

What does “the seq2seq model doesn't care about alignment” mean?

Alignment is the correspondence between target words and source words (padathtai  $\leftrightarrow$  movie). Vanilla seq2seq gives the decoder only  $h_5$ , so there is no mechanism linking an output word to particular input words. Attention creates this link through weights  $\alpha_{ti}$ .

**S3**

Explain the attention mechanism in seq2seq in your own words, with formulas.

Keep all encoder states  $h_1..h_n$ . At decoder step  $t$ , score each  $h_i$  against  $s_{t-1}$ , softmax the scores into  $\alpha_{ti}$ , and form  $c_t = \sum_i \alpha_{ti} h_i$ .  $c_t$  is a source summary tailored for output word  $t$  and is fed into the decoder.

**S4**

Why can the  $\alpha_{ti}$  at a step be computed in parallel but the  $h_i$  cannot?

At step  $t$  all  $h_i$  and  $s_{t-1}$  already exist, and each  $\alpha_{ti}$  needs only these, so they are independent. Each  $h_i$  however requires  $h_{i-1}$ : a chain of dependencies that forces sequential computation.

**S5**

State the “major limitation” of RNN seq2seq and the wishlist.

Given a training example, we cannot parallelise the sequence of computations across time steps, so training is slow and GPUs are underused. Wishlist: an architecture that incorporates attention, allows parallelisation, and removes vanishing/exploding gradients.

**S6**

Why are vanishing/exploding gradients a problem for RNNs but much less for the Transformer encoder?

In an RNN, the gradient from step  $t$  to step 1 passes through  $t - 1$  recurrent multiplications, which can shrink or grow exponentially. The Transformer has no recurrence through time; any two positions are connected directly by attention in a single layer, so the gradient path length does not grow with the sentence.

**S7**

Explain self-attention using the “it” example.

In “... because it was too tired”, “it” = animal; in “... because it was congested”, “it” = street. The right meaning depends on other words in the same sentence, so each word attends over its own sentence to build a context-dependent representation. Since the sequence attends to itself, it is self-attention.

**S8**

What is the “goal” of self-attention as stated in the lecture?

Given a word, compute relational scores between it and every word in the sentence, so that scores are higher for contextually related words. Collected together this is a  $T \times T$  table whose row  $i$  describes what word  $i$  attends to.

**S9**

In the score table, rows are treated as  $s_i$  and columns as  $h_j$ . Why does self-attention allow computing all rows in parallel while seq2seq did not?

In seq2seq,  $s_i$  came from the decoder RNN one at a time. In self-attention both row and column vectors come from the input embeddings, which are all available at once, so no row depends on another.

**S10**

Break down the seq2seq score function and explain why self-attention needs three matrices.

$V_a^T \tanh(U_{att}s_{t-1} + W_{att}h_j)$  uses three vectors, two linear transformations, one non-linearity and a final dot product. Self-attention receives only embeddings  $h_j$ , so three vectors (query, key, value) are made from each  $h_j$  using three learned matrices  $W_Q, W_K, W_V$ .

**S11**

Explain the roles of query, key and value.

Query: what the current word looks for. Key: how each word presents itself for matching. Value: the information a word contributes once it is attended to. Query-key dot products decide weights; weights combine values.

**S12**

Why not use the raw embedding for query, key and value?

Scores would be  $h_i^T h_j$ : symmetric and fixed (no parameters), with self-similarity  $\|h_i\|^2$  tending to dominate. Learned  $W_Q, W_K$  make scores asymmetric and trainable, and  $W_V$  lets the model choose what information to pass, separately from matching. Also they reduce the dimension from 512 to 64.

**S13**

Describe the computation of  $z_1$  step by step.

(1)  $q_j, k_j, v_j$  for all words. (2)  $e_{1j} = q_1 \cdot k_j$  for all  $j$ . (3)  $\alpha_{1j} = \text{softmax}(e_{1j})$ . (4)  $z_1 = \sum_j \alpha_{1j} v_j$ . Repeat with  $q_2$  for  $z_2$ , etc.

**S14**

How is self-attention vectorised? Give shapes.

Stack embeddings:  $H \in \mathbb{R}^{512 \times T}$ .  $Q = W_Q H$ ,  $K = W_K H$ ,  $V = W_V H$ , each  $64 \times T$ .  $Z = \text{softmax}(Q^T K / \sqrt{d_k}) V^T$ :  $Q^T K$  is  $T \times T$ ,  $Z$  is  $T \times 64$ . All outputs come from a few matrix multiplications, which GPUs run efficiently.

**S15**

Why is it called “scaled” dot-product attention? Derive why  $\sqrt{d_k}$ .

Because  $Q^T K$  is scaled by  $1/\sqrt{d_k}$ . With i.i.d. zero-mean unit-variance components,  $\text{Var}(q \cdot k) = d_k$ . Large scores saturate softmax (near one-hot, tiny gradients). Dividing by  $\sqrt{d_k}$  gives variance 1, keeping gradients healthy.

**S16**

Why use  $\sqrt{d_k}$  and not  $d_k$ ?

Variance scales by the square of the divisor. Dividing by  $\sqrt{d_k}$  turns variance  $d_k$  into 1. Dividing by  $d_k$  would give variance  $1/d_k$ , squashing scores toward 0 and making attention nearly uniform for large  $d_k$ .

**S17**

Explain the difference between the lecture’s formula and the paper’s.

Lecture (columns):  $\text{softmax}(Q^T K / \sqrt{d_k}) V^T$ . Paper (rows):  $\text{softmax}(Q K^T / \sqrt{d_k}) V$ . Only the vector orientation differs; the computation is identical.

**S18**

Motivate multi-head attention with the CNN analogy.

CNNs use many filters so each learns a different feature. Likewise several attention heads with separate  $W_{Q_i}, W_{K_i}, W_{V_i}$  learn different contextual relations. Like kernels, heads run independently, so they are computed in parallel.

**S19**

What empirical evidence supports multiple heads?

Tensor2Tensor visualisation: with one head, “it” attends strongly to “was”. With two heads, head 1 links “it” to “was” and head 2 links “it” to “animal”. Different heads capture different contextual information.

**S20**

Describe how multi-head outputs are combined, with shapes.

Each of 8 heads outputs  $T \times 64$ . They are concatenated to  $T \times 512$  and passed through a linear layer  $W_O \in \mathbb{R}^{512 \times 512}$ , giving  $T \times 512$ , the same shape as the input, so layers can be stacked.

**S21**

Is multi-head attention much more expensive than single-head? Explain.

No. Total projection parameters are  $3h \cdot (d_{\text{model}}/h) \cdot d_{\text{model}} = 3d_{\text{model}}^2$  regardless of  $h$ , because each head works in a smaller  $d_k = d_{\text{model}}/h$  space. We get multiple attention patterns for about the same cost as one full-width head.

**S22**

What is an encoder layer? What is the encoder?

An encoder layer has two sub-layers: multi-head attention and a position-wise FFN. The encoder is a stack of  $N = 6$  such layers with identical structure but separate weights.

**S23**

Explain the position-wise FFN and why it is needed.

$\text{FFN}(z) = \max(0, W_1 z + b_1) W_2 + b_2$ ,  $512 \rightarrow 2048 \rightarrow 512$ , applied to each position with the same weights. Attention outputs are weighted averages (largely linear in the values); the FFN adds non-linearity and per-position processing capacity. Weight sharing across positions keeps parameters independent of  $T$  and lets positions run in parallel.

**S24**

“Parallel horizontally, not vertically.” Explain.

Within a layer, all  $T$  positions are computed simultaneously (horizontal). Layers depend on the previous layer’s output, so Layer-2 waits for Layer-1 (vertical). The number of sequential steps is  $N$ , not  $T$ .

**S25**

Justify the “about 18 million parameters” figure.

Per layer: attention  $\approx 1.05\text{M}$  ( $4 \times 512^2$ ) and FFN  $\approx 2.1\text{M}$  ( $2 \times 512 \times 2048$ ), so  $\approx 3\text{M}$ . With  $N = 6$ :  $\approx 18\text{M}$  (18.9M with biases).

**S26**

Compare self-attention and cross-attention.

Self:  $Q, K, V$  from the same sequence (encoder words attending to encoder words). Cross (encoder-decoder):  $Q$  from the decoder,  $K, V$  from encoder outputs  $e_j$ . Cross-attention plays the role of seq2seq alignment; its matrix is  $m \times T$ .

**S27**

What does the encoder output, and how is it used?

A sequence  $e_1, \dots, e_T$ , one 512-dim contextual vector per input word. The decoder's encoder-decoder attention uses them as keys and values.

**S28**

What is the main computational drawback of self-attention?

It compares every pair of positions: time and memory for the attention matrix grow as  $O(T^2)$ . Very long sequences become expensive (e.g.  $T = 1024$ , 8 heads: 32 MiB of weights per layer in fp32).

**S29**

Walk through the full journey of the word “it” through one encoder layer.

Embedding  $h_{it} \in \mathbb{R}^{512} \rightarrow$  in each of 8 heads:  $q_{it}, k_{it}, v_{it} \in \mathbb{R}^{64}$ ;  $q_{it}$  is scored against all keys, scaled, softmaxed; weighted sum of values gives a 64-dim output (head 1 may focus on “was”, head 2 on “animal”)  $\rightarrow$  concat to 512  $\rightarrow W_O \rightarrow z_{it} \in \mathbb{R}^{512} \rightarrow$  FFN ( $512 \rightarrow 2048 \rightarrow 512$ )  $\rightarrow o_{it}$ , a context-aware vector for “it”, input to the next layer.

**S30**

Does self-attention by itself know word order? (Think about it.)

No. The computation is a set of dot products and weighted sums; shuffling input words just shuffles the outputs in the same way (permutation-equivariant). Nothing in Week 1's formulas encodes position, so extra positional information must be added to embeddings. This is a natural next topic.

## 7 Multiple Choice Questions (single correct)

### MCQ 1

The final hidden state of the encoder RNN is NOT called:

- (A) concept vector
- (B) thought vector
- (C) query vector
- (D) annotation

(C). Query vectors belong to self-attention.

### MCQ 2

In seq2seq with attention, the decoder's initial state is:

- (A)  $h_0$
- (B)  $h_n$  (the final encoder state)
- (C) the average of all  $h_i$
- (D) a random vector

(B)  $s_0 = h_5$  in the example.

### MCQ 3

The context vector  $c_t$  equals:

- (A)  $\sum_t \alpha_{ti} h_i$
- (B)  $\sum_i \alpha_{ti} h_i$
- (C)  $\sum_i \alpha_{ti} s_i$
- (D)  $h_t$

(B). Sum over source positions  $i$ .

### MCQ 4

The attention weights at decoder step  $t$  satisfy:

- (A)  $\sum_t \alpha_{ti} = 1$
- (B)  $\sum_i \alpha_{ti} = 1$  and  $\alpha_{ti} > 0$
- (C)  $\alpha_{ti} \in \{0, 1\}$
- (D) they can be negative

(B). Softmax over  $i$ .

**MCQ 5**

In  $\alpha_{ti} = \text{align}(y_t, h_i)$ , the score is computed between:

- (A)  $s_t$  and  $h_i$
- (B)  $s_{t-1}$  and  $h_i$
- (C)  $y_t$  and  $s_t$
- (D)  $h_{i-1}$  and  $h_i$

(B). The previous decoder state.

**MCQ 6**

Which is true for RNN seq2seq with attention?

- (A) Encoder states can be computed in parallel
- (B) Attention weights at a step can be computed in parallel
- (C) Nothing can be parallelised
- (D) Decoder states can be computed in parallel

(B).

**MCQ 7**

The main training limitation of RNN seq2seq stressed in the lecture is:

- (A) Too many parameters
- (B) Cannot parallelise computations across time steps
- (C) Cannot use attention
- (D) Cannot handle variable length

(B).

**MCQ 8**

The Transformer paper is titled:

- (A) Sequence to Sequence Learning
- (B) Neural Machine Translation by Jointly Learning to Align
- (C) Attention Is All You Need
- (D) Deep Residual Learning

(C).

**MCQ 9**

In the Transformer encoder the order of blocks is:

- (A) Self-attention → Embedding → FFN
- (B) Embedding → Self-attention → FFN
- (C) Embedding → FFN → Self-attention
- (D) FFN → Embedding → Self-attention

(B).

**MCQ 10**

Which block appears in the decoder but not the encoder?

- (A) Self-attention
- (B) Feed forward network
- (C) Encoder–decoder attention
- (D) Word embedding

(C).

**MCQ 11**

In “The animal didn’t cross the street because it was congested”, “it” refers to:

- (A) animal
- (B) street
- (C) cross
- (D) because

(B).

**MCQ 12**

Self-attention is called “self” because:

- (A) it uses no parameters
- (B) a sequence attends to itself
- (C) it is unsupervised
- (D) it only attends to the same word

(B).

**MCQ 13**

The seq2seq additive score  $V_a^T \tanh(Us + Wh)$  contains how many non-linearities?

- (A) 0
- (B) 1
- (C) 2
- (D) 3

**(B)** one (tanh), with two linear transformations.

**MCQ 14**

How many transformation matrices does a single self-attention head need?

- (A) 1
- (B) 2
- (C) 3
- (D) 4

**(C)**  $W_Q, W_K, W_V$ .

**MCQ 15**

The elements of  $W_Q, W_K, W_V$  are:

- (A) fixed to identity
- (B) random and never updated
- (C) learned during training
- (D) computed from the softmax

**(C)**.

**MCQ 16**

The score function used in Transformer self-attention is:

- (A) additive (tanh)
- (B) (scaled) dot product
- (C) cosine distance
- (D) Euclidean distance

**(B)**.

**MCQ 17**

$e_{1j}$  is:

- (A)  $[q_1 \cdot k_1, \dots, q_1 \cdot k_T]$
- (B)  $[k_1 \cdot v_1, \dots]$
- (C)  $[q_1 \cdot q_j]$
- (D)  $[v_1 \cdot k_j]$

(A).

**MCQ 18**

$z_1$  equals:

- (A)  $\sum_j \alpha_{1j} k_j$
- (B)  $\sum_j \alpha_{1j} q_j$
- (C)  $\sum_j \alpha_{1j} v_j$
- (D)  $\sum_j \alpha_{j1} v_j$

(C).

**MCQ 19**

To compute  $z_2$  we need:

- (A)  $z_1$  first
- (B)  $q_2$  and all keys and values
- (C) only  $k_2$  and  $v_2$
- (D)  $h_1$  only

(B). No dependence on  $z_1$ .

**MCQ 20**

With  $H \in \mathbb{R}^{512 \times T}$  and  $Q = W_Q H \in \mathbb{R}^{64 \times T}$ ,  $W_Q$  has shape:

- (A)  $512 \times 64$
- (B)  $64 \times 512$
- (C)  $64 \times T$
- (D)  $T \times 64$

(B).

**MCQ 21**

Shape of  $Q^T K$  is:

- (A)  $64 \times 64$
- (B)  $T \times T$
- (C)  $T \times 64$
- (D)  $512 \times T$

(B).

**MCQ 22**

Shape of the single-head output  $Z$  is:

- (A)  $T \times T$
- (B)  $T \times 64$
- (C)  $64 \times 64$
- (D)  $T \times 512$

(B).

**MCQ 23**

The softmax in  $\text{softmax}(Q^T K / \sqrt{d_k})$  is applied:

- (A) over the whole matrix
- (B) column-wise
- (C) row-wise (per query)
- (D) element-wise

(C). Each query's weights over keys sum to 1.

**MCQ 24**

The purpose of dividing by  $\sqrt{d_k}$  is to:

- (A) reduce parameters
- (B) prevent softmax saturation/vanishing gradients
- (C) make attention symmetric
- (D) enforce sparsity

(B).

**MCQ 25**

If  $q, k$  components are i.i.d.  $(0, 1)$  with  $d_k = 64$ ,  $\text{Var}(q \cdot k)$  is:

- (A) 1
- (B) 8
- (C) 64
- (D) 4096

(C).

**MCQ 26**

In the original paper's row convention the formula is:

- (A)  $\text{softmax}(Q^T K / \sqrt{d_k}) V^T$
- (B)  $\text{softmax}(Q K^T / \sqrt{d_k}) V$
- (C)  $\text{softmax}(Q K / d_k) V$
- (D)  $\text{softmax}(K^T Q) V$

(B).

**MCQ 27**

Order of operations in a scaled dot-product head:

- (A) Softmax, Scale, MatMul, MatMul
- (B) MatMul, Scale, Softmax, MatMul
- (C) Scale, MatMul, MatMul, Softmax
- (D) MatMul, Softmax, Scale, MatMul

(B).

**MCQ 28**

The CNN analogue of an attention head is:

- (A) a pooling layer
- (B) a filter/kernel
- (C) a stride
- (D) padding

(B).

**MCQ 29**

In the two-head visualisation, head 2 links “it” most strongly to:

- (A) was
- (B) street
- (C) animal
- (D) tired

(C); head 1 links it to “was”.

**MCQ 30**

In the original Transformer the number of heads is:

- (A) 4
- (B) 6
- (C) 8
- (D) 12

(C).

**MCQ 31**

After concatenating 8 heads, each  $T \times 64$ , the shape is:

- (A)  $T \times 64$
- (B)  $T \times 512$
- (C)  $8 \times T$
- (D)  $512 \times 512$

(B).

**MCQ 32**

$W_O$  has shape:

- (A)  $64 \times 512$
- (B)  $512 \times 512$
- (C)  $512 \times 2048$
- (D)  $T \times 512$

(B).

**MCQ 33**

Doubling the number of heads from 8 to 16 with  $d_{\text{model}} = 512$  fixed makes  $d_k$ :

- (A) 128
- (B) 64
- (C) 32
- (D) 16

(C).

**MCQ 34**

Each encoder layer has how many sub-layers?

- (A) 1
- (B) 2
- (C) 3
- (D) 6

(B) multi-head attention and FFN.

**MCQ 35**

The number of encoder layers  $N$  in the original Transformer is:

- (A) 2
- (B) 4
- (C) 6
- (D) 12

(C).

**MCQ 36**

The FFN hidden layer width is:

- (A) 64
- (B) 512
- (C) 1024
- (D) 2048

(D).

**MCQ 37**

The activation in the FFN is:

- (A) tanh
- (B) sigmoid
- (C) ReLU
- (D) softmax

(C)  $\max(0, \cdot)$ .

**MCQ 38**

“Position-wise” FFN means:

- (A) different FFN per position
- (B) same FFN applied independently to each position
- (C) FFN mixes positions
- (D) FFN only on the first position

(B).

**MCQ 39**

Where do different positions exchange information in an encoder layer?

- (A) Only in the FFN
- (B) Only in multi-head attention
- (C) In both
- (D) Nowhere

(B).

**MCQ 40**

Approximate parameters in one original encoder layer:

- (A) 0.3M
- (B) 1M
- (C) 3M
- (D) 18M

(C).

**MCQ 41**

Approximate parameters in the full 6-layer encoder:

- (A) 3M
- (B) 6M
- (C) 18M
- (D) 65M

(C).

**MCQ 42**

Which sub-layer holds more parameters in the original layer?

- (A) Multi-head attention
- (B) FFN
- (C) Equal
- (D) Softmax

(B) about 2.1M vs 1.05M.

**MCQ 43**

The encoder stack is parallelised:

- (A) vertically across layers
- (B) horizontally across positions
- (C) both
- (D) neither

(B).

**MCQ 44**

Input and output shapes of each encoder layer are:

- (A)  $T \times 512$  and  $T \times 64$
- (B)  $T \times 512$  and  $T \times 512$
- (C)  $T \times 64$  and  $T \times 512$
- (D)  $T \times 2048$  and  $T \times 512$

(B).

**MCQ 45**

Encoder outputs are denoted  $e_j$  for  $j = 1, \dots, ?$

- (A)  $N$
- (B)  $T$
- (C)  $d_k$
- (D) 8

(B) one per input position.

**MCQ 46**

In cross-attention, the keys and values come from:

- (A) the decoder
- (B) the encoder outputs
- (C) the word embeddings of the target
- (D)  $W_O$

(B).

**MCQ 47**

Time complexity of computing one head's  $Q^T K$ :

- (A)  $O(T)$
- (B)  $O(Td_k)$
- (C)  $O(T^2d_k)$
- (D)  $O(Td_k^2)$

(C).

**MCQ 48**

Max path length between two positions in one self-attention layer:

- (A)  $O(1)$
- (B)  $O(\log T)$
- (C)  $O(T)$
- (D)  $O(T^2)$

(A).

**MCQ 49**

If all scores for a query are equal,  $z$  is:

- (A) zero
- (B) the value of the first word
- (C) the average of all values
- (D) undefined

(C).

**MCQ 50**

$\text{softmax}([1, 2, 3])$  vs  $\text{softmax}([101, 102, 103])$ :

- (A) the second is sharper
- (B) they are identical
- (C) the first is sharper
- (D) the second overflows to zero

(B) shift invariance.

**MCQ 51**

Which is NOT a reason for using  $W_Q \neq W_K$ ?

- (A) Allows asymmetric relations
- (B) Makes scores learnable
- (C) Guarantees each word attends most to itself
- (D) Separates “looking for” from “offering”

(C). It guarantees no such thing.

**MCQ 52**

Tamil translation alignment in the lecture is:

- (A) strictly diagonal
- (B) not diagonal (re-ordered)
- (C) always the identity
- (D) random

(B).

**MCQ 53**

In the lecture example, the embedding of “transformers” is:

- (A)  $[1, 0, 0]$
- (B)  $[1, 0, 1]$
- (C)  $[1, 1, 1]$
- (D)  $[0, 1, 0]$

(C).

## 8 Multiple Select Questions (one or more correct)

### MSQ 1

Names for the final encoder state include:

- (A) concept vector
- (B) thought vector
- (C) context vector
- (D) annotation

A, B, C, D.

### MSQ 2

Limitations of RNN seq2seq models:

- (A) Cannot parallelise across time steps in training
- (B) Vanishing/exploding gradients
- (C) No alignment without attention
- (D) Cannot produce variable-length outputs

A, B, C.

### MSQ 3

Which can be computed in parallel?

- (A) All  $\alpha_{ti}$  for a fixed  $t$  in seq2seq
- (B) All encoder RNN states
- (C) All  $z_i$  in one self-attention layer
- (D) All heads in multi-head attention

A, C, D.

### MSQ 4

The wishlist for a new architecture included:

- (A) incorporate attention
- (B) allow parallelisation
- (C) remove vanishing/exploding gradients
- (D) use more recurrence

A, B, C.

**MSQ 5**

Components of a Transformer decoder layer (per the lecture diagram):

- (A) Self-attention
- (B) Encoder–decoder attention
- (C) Feed forward network
- (D) RNN cell

**A, B, C.**

**MSQ 6**

Which are learned parameters?

- (A)  $W_Q$
- (B)  $W_O$
- (C)  $\sqrt{d_k}$
- (D)  $W_1, b_1$

**A, B, D.**  $\sqrt{d_k}$  is a fixed constant.

**MSQ 7**

Correct shape statements ( $d_{\text{model}} = 512$ , 8 heads):

- (A)  $Q \in \mathbb{R}^{64 \times T}$
- (B)  $Q^T K \in \mathbb{R}^{T \times T}$
- (C)  $Z_{\text{head}} \in \mathbb{R}^{T \times 64}$
- (D)  $W_O \in \mathbb{R}^{64 \times 64}$

**A, B, C.**  $W_O$  is  $512 \times 512$ .

**MSQ 8**

Properties of softmax:

- (A) outputs sum to 1
- (B) invariant to adding a constant
- (C) invariant to multiplying by a constant
- (D) order-preserving

**A, B, D.**

**MSQ 9**

Effects of dividing scores by  $\sqrt{d_k}$ :

- (A) variance of scores becomes about 1
- (B) softmax becomes less peaky
- (C) gradients are less likely to vanish
- (D) number of parameters decreases

**A, B, C.**

**MSQ 10**

True statements about multi-head attention:

- (A) Each head has its own  $W_Q, W_K, W_V$
- (B) Heads are concatenated then projected by  $W_O$
- (C) All heads share the same attention weights
- (D) Different heads can capture different relations

**A, B, D.**

**MSQ 11**

True about the position-wise FFN:

- (A) Same weights for all positions
- (B) Hidden width 2048
- (C) Uses ReLU
- (D) Mixes information across positions

**A, B, C.**

**MSQ 12**

True about the encoder stack:

- (A)  $N = 6$  identical-structure layers
- (B) Each layer has its own weights
- (C) Layers computed in parallel with each other
- (D)  $\approx 18\text{M}$  parameters

**A, B, D.**

**MSQ 13**

Self-attention vs cross-attention:

- (A) Self:  $Q, K, V$  from one sequence
- (B) Cross:  $Q$  from decoder,  $K, V$  from encoder
- (C) Cross-attention matrix must be square
- (D) Both use scaled dot product

**A, B, D.**

**MSQ 14**

The seq2seq additive score involves:

- (A) two linear transformations
- (B) one non-linearity
- (C) a final dot product with  $V_a$
- (D) a softmax inside the tanh

**A, B, C.**

**MSQ 15**

Which grow quadratically with sequence length  $T$ ?

- (A) size of the attention matrix
- (B) cost of  $Q^T K$
- (C) number of parameters in  $W_Q$
- (D) cost of computing  $Q = W_Q H$

**A, B.** Parameters are independent of  $T$ ;  $W_Q H$  is linear in  $T$ .

## 9 True or False (with justification)

**TF1.** The decoder in vanilla seq2seq can look at individual source words.

*Answer:* False. It sees only  $h_n$ .

**TF2.** Attention weights at a decoder step can be computed in parallel.

*Answer:* True.

**TF3.** RNN encoder hidden states can be computed in parallel.

*Answer:* False.  $h_i$  depends on  $h_{i-1}$ .

**TF4.** The Transformer removes recurrence completely.

*Answer:* True. It relies on attention and FFNs.

**TF5.** In self-attention, the query of a word is compared with the keys of all words, including itself.

*Answer:* True.

**TF6.** A word always attends most strongly to itself.

*Answer:* False. See N17, where “I” attends least to itself.

**TF7.**  $q_i \cdot k_j = q_j \cdot k_i$  in general.

*Answer:* False. Different  $W_Q, W_K$  break symmetry (N13: 0.2115 vs 0.20).

**TF8.**  $W_Q$  is different for each word position.

*Answer:* False. Shared across positions (different per head and per layer).

**TF9.** Dividing by  $\sqrt{d_k}$  changes which key gets the highest weight.

*Answer:* False. Positive scaling preserves order; it changes only sharpness.

**TF10.** Softmax is invariant to multiplying all scores by 2.

*Answer:* False. It gets sharper.

**TF11.**  $d_k = d_{\text{model}}/h$  in the original Transformer.

*Answer:* True:  $512/8 = 64$ .

**TF12.** Multi-head attention has 8 times the projection parameters of a single 512-dim head.

*Answer:* False. Same  $3d_{\text{model}}^2$  total.

**TF13.** The output of multi-head attention has the same shape as its input.

*Answer:* True:  $T \times 512$ .

**TF14.** The FFN has different weights at each position.

*Answer:* False. Position-wise = shared.

**TF15.** The FFN has different weights in different layers.

*Answer:* True. Each layer has its own parameters.

**TF16.** Positions interact inside the FFN.

*Answer:* False. Only in attention.

**TF17.** Encoder layers can be computed simultaneously.

*Answer:* False. Layer  $l$  needs Layer  $l - 1$ .

**TF18.** The encoder has about 18 million parameters.

*Answer:* True ( $6 \times \approx 3\text{M}$ ).

**TF19.** In cross-attention the attention matrix is always square.

*Answer:* False.  $m \times T$ .

**TF20.** Self-attention cost grows linearly with  $T$ .

*Answer:* False. Quadratically.

**TF21.** Seq2seq alignment learned by attention must be monotonic.

*Answer:* False. Tamil example re-orders words.

**TF22.** The number of parameters of a self-attention layer depends on  $T$ .

*Answer:* False.

**TF23.** The encoder output has one vector per input word.

*Answer:* True:  $e_1, \dots, e_T$ .

**TF24.** Without the ReLU, stacking attention+FFN layers still gives a highly non-linear model.

*Answer:* False, the FFN would become linear; softmax provides some non-linearity in the weights but the value path is linear.

**TF25.** Self-attention on its own is sensitive to word order.

*Answer:* False. It is permutation-equivariant.

## 10 Fill in the Blanks

1. The final encoder state is called the concept / thought / context vector or annotation.
2. In seq2seq, the decoder initial state  $s_0$  equals  $h_n$  ( $h_5$ ).
3.  $c_t = \sum_{i=1}^n \alpha_{ti} h_i$ .
4. The attention weights are obtained by applying softmax to the scores.
5. The seq2seq score is  $V_a^T \tanh(U_{att} s_{t-1} + W_{att} h_j)$ .
6. Attention solves the problem of alignment between source and target words.
7. The Transformer paper is titled “Attention Is All You Need”.
8. In self-attention, the three vectors for each word are query, key, value.
9.  $k_j = W_K h_j$ .
10. The self-attention score function is the dot product.
11.  $Z = \text{softmax}(Q^T K / \sqrt{d_k}) V^T$ .
12.  $Q^T K$  has shape  $T \times T$ .
13.  $d_k$  in the original Transformer is 64.
14. The number of heads is 8.
15. Concatenated heads have shape  $T \times 512$ , projected by  $W_O$ .
16. The CNN analogue of an attention head is a filter / kernel.
17. Each encoder layer has 2 sub-layers; the encoder has 6 layers.
18.  $\text{FFN}(z) = \max(0, W_1 z + b_1) W_2 + b_2$  with hidden width 2048.
19. The encoder has about 18 million parameters.
20. The encoder is parallelised in the horizontal direction, not the vertical.
21. In cross-attention, queries come from the decoder, keys/values from the encoder.
22. The variance of  $q \cdot k$  with i.i.d. unit-variance components is  $d_k$ .

## 11 Quick Self-Check Before the Exam

You are ready if you can, without notes:

1. Draw the seq2seq encoder–decoder and mark  $h_i$ ,  $s_i$ ,  $s_0 = h_5$ .
2. Write  $c_t$  and  $\alpha_{ti}$  and say which parts are parallel.
3. State the major limitation and the three-item wishlist.
4. Explain the “it” example and why self-attention is needed.
5. Write  $q_j, k_j, v_j$  and explain why 3 learned matrices.
6. Compute a small  $z_1$  by hand (dot products  $\rightarrow$  softmax  $\rightarrow$  weighted sum).
7. Write  $Z = \text{softmax}(Q^T K / \sqrt{d_k}) V^T$  with every shape, and the paper’s version.

8. Derive why  $\sqrt{d_k}$ .
9. Explain multi-head attention, the CNN analogy, the “was”/“animal” heads, concat and  $W_O$ .
10. Write the FFN formula and explain “position-wise”.
11. Count parameters:  $\approx 1\text{M}$  attention +  $\approx 2\text{M}$  FFN =  $3\text{M}$  per layer,  $18\text{M}$  total.
12. Explain “horizontal, not vertical” parallelism and self vs. cross attention.